Storing a key is pretty similar to checking it, only we need a little extra effort to keep track of the number of elements added to the filter, and to use `write` instead of `read`. Figure 4.5 shows a step by step example, putting together all the little pieces we have coded so far.

Note that in this implementation for `insert`, shown in listing 4.11, when we compute the size of the filter, we keep track of the number of unique elements added to the filter, rather than the total number of times the `add` method is called.

Listing 4.11 Method insert

```
function
insert(key)
      #1
  positions ←
key2Positions(key)
  #2
  if not contains(key, positions)
then                                          #3
    this.size ← this.size + 1
    positions.map((i) => writeBit(this.bitsArray,
i));                          #4
```

#1 Function `insert` takes a key and stores it into the Bloom filter.

#2 Transforms the string representation of the key to add into a sequence of `k` bit indices.

#3 Before incrementing the size of the filter and storing 1 in each of the bit corresponding to the bits, we check that the key is not already contained by the filter. This is more than just optimization: `size` is crucial to estimate the